



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Recognized as Deemed to be University under Section 3 of UGC Act 1956)

AI-BASED CONSTRAINT SATISFACTION SOLVER FOR AUTOMATED TIMETABLE SCHEDULING

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

Course Code & Course Name

to the award of the degree of

BACHELOR OF ENGINEERING

IN

ECE, AIDS, CSE

Submitted by

K.S.Dharshana (192412015)

Kaviya.G (192524281)

Vidhushini.T.S (192511061)

Under the Supervision of

Dr. Rajaram.P

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “AI – Based Constraint Satisfaction Solver for Automated Timetable Scheduling” has been carried out by K.S.Dharshana (192412015) , G.Kaviya (192524218) & T.S.Vidhushini (19251101) under the supervision of Dr.Rajaram.P and is submitted in partial fulfilment of the requirements for the current semester of the B.E ECE, B.E AIDS & B.E CSE program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

**Dr. Krishnakumar Kathiresan,
Professor
CSE/ Generative-AI
SIMATS Engineering,
SIMATS, Chennai – 602105.**

COURSE FACULTY

**Dr. Rajaram.P,
Professor
CSE / Data Vista
SIMATS Engineering,
SIMATS, Chennai – 602105.**

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

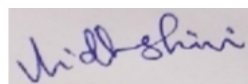
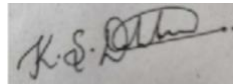
DECLARATION

We, K.S.Dharshana (192412015), G.Kaviya(192524281) and T.S.Vidhushini (192511061) of the ECE,AIDS & CSE , SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “AI- Based Constraint Satisfaction Solver for Automated Timetable Scheduling ” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Students with Names



Individual Contribution Statement

(Mandatory for all team projects — percentages below are a proposed working split aligned to the three project modules; the team should confirm/adjust these figures before final submission.)

Student Name / Reg. No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Contribution (%)
T. S. Vidhushini 192511061	Module 1 – Constraint modelling and data-collection interface	Data Management UI (Add Subject/Faculty/Room), input validation	Unit testing of data-entry forms and constraint model build-up	Ch.1, Ch.3 (3.1–3.3)	34%
K. S. Dharshana 192412015	Module 3 – Timetable generation and conflict-reporting interface	Dashboard, generated-timetable view, constraint status panel	End-to-end verification of generated timetables against constraints	Ch.3 (3.4–3.7), Ch.4	33%
G. Kaviya 192524281	Module 2 – CSP solving engine (backtracking / local search)	Backtracking solver, MRV heuristic, forward checking logic	Performance testing (generation time, optimality, conflict count)	Ch.2, Ch.5	33%

Total: 100%

Abstract

Timetable construction for an engineering department is a classic example of a combinatorially explosive scheduling problem: every subject session must be assigned a day, a period and a room such that no faculty member, room or student section is double-booked, and every subject receives its required number of weekly periods within a fixed slot grid. When performed manually, this task is time-consuming, error-prone and frequently produces clashes that surface only after publication, disrupting both faculty and student schedules. This project addresses the engineering problem of generating a conflict-free, optimal class timetable automatically, formulated as a Constraint Satisfaction Problem (CSP).

The proposed solution, the AI Scheduler, is a web-based system organised into three modules: (i) a data-collection module that captures subjects, faculty availability, room capacity/type and the institutional time-slot grid; (ii) a CSP solving engine that models faculty-conflict, room-conflict, room-capacity, time-availability, weekly-hours and section-conflict rules as hard constraints and searches the assignment space using backtracking search with a minimum-remaining-values (MRV) heuristic, forward checking, and provision for local-search refinement (simulated annealing) when the search space grows; and (iii) a timetable-generation and conflict-reporting interface that renders the resulting schedule and displays the status of every constraint.

The methodology followed a build-test-verify cycle: constraints were first encoded and validated individually, then the solver was run on a representative department-scale dataset of four subjects, four faculty members and three rooms across a 5-day, 5-period (25-slot) grid. The system consistently returned an OPTIMAL solution, generating all twelve required class sessions in 0.357 seconds with zero constraint violations across all six active constraint categories. These results confirm that a heuristic backtracking CSP formulation is well suited to department-scale academic timetabling, delivering both correctness (zero conflicts) and practicality (sub-second generation time) without requiring specialised optimisation infrastructure.

The principal conclusion is that automated, constraint-driven scheduling can reliably replace manual timetable preparation at this scale while remaining transparent and auditable to faculty, and that the architecture readily extends toward larger, multi-department deployments through local-search-based optimisation and persistent data storage.

Keywords: Constraint Satisfaction Problem, Automated Timetabling, Backtracking Search, Heuristic Optimisation, Conflict-Free Scheduling, AI Scheduler

Table of Contents

Sl. No.	Title	Page No.
i	Certificate from the Supervisor	ii
ii	Declaration by the Candidate	iii
iii	Individual Contribution Statement	iv
iv	Abstract	v
v	List of Figures	vi
vi	List of Tables	vii
vii	List of Abbreviations	viii
1	Chapter 1: Introduction and Engineering Problem	8
2	Chapter 2: Literature Review and Existing Solutions	12
3	Chapter 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	15
4	Chapter 4: Implementation, Testing & Results, Project Management	24
5	Chapter 5: Conclusion, Future Work and Reflection	31
	References	33
	Appendices	35
	Capstone Project Outcome Mapping Sheet	37

List of Figures

Figure No.	Figure Name	Page No.
Fig. 3.1	System Architecture / Module Flow Diagram	19
Fig. 4.1	AI Scheduler – Data Management Interface	25

List of Tables

Table No.	Table Name	Page No.
Table 2.1	Comparative Analysis of Existing Timetabling Approaches	13
Table 3.1	Stakeholder / User Requirements	15
Table 3.2	Functional and Non-Functional Requirements	15
Table 3.3	Applicable Constraints and Standards	17
Table 3.4	Design Specifications	17
Table 3.5	Evaluation of Alternative Solutions	18
Table 3.6	Trade-off Analysis	20
Table 3.7	Sustainability, Ethics, Safety and Security Considerations	21
Table 3.8	Risk Register	22
Table 4.1	Tools, Software and Equipment Used	25
Table 4.2	Test Plan and Verification	26
Table 4.3	Achieved Results vs. Required Values	26
Table 4.4	Achievement of Objectives	28
Table 4.5	Project Milestones	29
Table 4.6	Roles and Contribution	30
Table 4.7	Estimated vs. Actual Cost	30

List of Abbreviations / Symbols

Symbol	Description	Unit
AI	Artificial Intelligence	—
CSP	Constraint Satisfaction Problem	—
MRV	Minimum Remaining Values (heuristic)	—
GA	Genetic Algorithm	—
ILP	Integer Linear Programming	—
UI	User Interface	—
DB	Database	—
LMS	Learning Management System	—
s	Seconds (generation time)	sec

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Academic timetabling is the process of assigning a fixed set of teaching sessions to a limited grid of days, periods and rooms so that every subject receives its required contact hours without any faculty member, room or student section being double-booked. In an engineering department such as Electronics and Communication Engineering at SIMATS, this involves reconciling subject-specific weekly-hour requirements, faculty availability, room capacity/type (classroom vs. laboratory) and a fixed institutional slot grid (typically five working days with five periods each, i.e. 25 slots per week). The underlying assignment problem is a well-known instance of the Constraint Satisfaction Problem (CSP) and, in its general form, is NP-hard: the number of possible day-time-room assignments grows combinatorially with the number of subjects, faculty and rooms, making exhaustive manual search impractical beyond a handful of subjects.

1.2 Need for the Project

Manually prepared timetables at department scale routinely take several days of coordinator effort and still leave residual clashes — a faculty member scheduled for two sections at once, a laboratory booked beyond its capacity, or a subject falling short of its weekly-hour requirement. These errors surface late, typically after the timetable has already been circulated, and require repeated manual correction. The people directly affected are class coordinators (who bear the preparation burden), faculty (who face last-minute reassignment), and students (whose sections may be double-booked or left with irregular contact hours). Existing departmental practice relies on spreadsheet-based, trial-and-error allocation with no systematic conflict verification, which does not scale as the number of subjects, sections and faculty grows. There is, therefore, a clear engineering need for a system that encodes scheduling rules explicitly and searches for a provably conflict-free assignment automatically.

1.3 Problem Statement

Given a set of subjects, each requiring a fixed number of periods per week and taught by a designated faculty member to a defined student section of known strength; a set of rooms, each with a fixed capacity and type (classroom or laboratory); and a fixed weekly slot grid of five days and five periods (25 slots), the problem is to assign every required subject session to a (day, period, room) triple such that:

- no faculty member is assigned to two sessions in the same slot (faculty-conflict constraint);
- no room is assigned to two sessions in the same slot (room-conflict constraint);
- the assigned room's capacity is not exceeded by the section's student strength (room-capacity constraint);
- sessions are placed only within slots the faculty has declared as available (time-availability constraint);
- every subject receives exactly its required number of weekly periods (weekly-hours constraint); and
- no student section is assigned to two sessions in the same slot (section-conflict constraint).

This problem is engineering-relevant because the six constraints interact and can conflict with one another under a fixed slot budget — for example, satisfying every subject's weekly-hour requirement while respecting limited room and faculty availability is not always trivially feasible, and the solver must either find a feasible assignment or clearly report which constraint(s) prevent one.

1.4 Project Aim

To design and implement an AI-based Constraint Satisfaction Problem solver that automatically generates a conflict-free, optimised class timetable for an engineering department, replacing manual, error-prone scheduling with a transparent, rule-driven, web-based system.

1.5 Project Objectives

1. To design a formal constraint model covering faculty, room, capacity, availability, weekly-hours and section-conflict rules.
2. To develop a data-collection interface for entering subjects, faculty, rooms and the time-slot grid.
3. To implement a backtracking CSP solving engine with heuristic variable ordering (MRV) and forward checking.
4. To incorporate local-search-based refinement (simulated annealing) for scaling beyond small, fully-constrained instances.
5. To implement automatic conflict detection and a constraint-status reporting dashboard.
6. To test the solver on a representative department-scale dataset and evaluate generation time, conflict count and solution optimality.

1.6 Scope

Included: intra-department timetabling for a single institution, covering faculty-conflict, room-conflict, room-capacity, time-availability, weekly-hours and section-conflict constraints within a fixed 25-slot weekly grid, delivered as a browser-based dashboard (Data Management, Constraints, Generate, Reports).

Excluded: inter-department elective clashes, multi-campus or multi-shift scheduling, real-time dynamic rescheduling in response to ad hoc leave/cancellation, and integration with an institutional Learning Management System.

Assumptions: the weekly slot grid is fixed at five days and five periods; subject, faculty and room data are entered once per generation cycle and remain static during solving; each subject session occupies exactly one slot.

Boundary conditions: the solver is evaluated on department-scale inputs (of the order of tens of subjects/faculty/rooms); very large, multi-department instances are identified in Section 5.2 as future work requiring additional optimisation techniques.

1.7 Expected Engineering Outcome

The intended outcome is a working software prototype — the AI Scheduler — comprising a data-management module, a CSP-based solving engine, and a timetable-generation/reporting interface, capable of producing an optimal, conflict-free weekly timetable for a representative department dataset within a few seconds, together with a constraint-satisfaction status report confirming compliance with all active scheduling rules.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature and existing tools were identified through a combination of foundational computer-science references on timetabling complexity, established constraint-satisfaction and AI textbooks, open-source constraint-programming libraries (python-constraint, Google OR-Tools), and widely used academic/commercial timetabling software (e.g., FET — Free Timetabling Software, aSc TimeTables). The review focused on how each approach models scheduling constraints and searches the assignment space.

2.2 Review of Existing Work

Timetabling is formally established as a computationally hard scheduling problem, closely related to graph-colouring and multi-commodity flow formulations; this underlies why exhaustive manual or brute-force approaches do not scale [1]. Classical AI treatments of Constraint Satisfaction Problems describe backtracking search together with heuristics such as minimum-remaining-values (MRV) variable ordering and forward checking/arc-consistency as effective general-purpose techniques for problems of this structure [2], and these form the theoretical basis of the solving engine implemented in this project.

Beyond pure backtracking, the wider literature on automated timetabling surveys metaheuristic approaches — genetic algorithms, simulated annealing and tabu search — which trade solving-time guarantees for the ability to handle much larger, softly-constrained problem instances by accepting near-optimal rather than strictly optimal solutions [6]. Open-source constraint solvers such as Google's OR-Tools provide production-grade CP-SAT solvers capable of handling both the hard constraints modelled here and additional soft objectives at larger scale [3], while lightweight libraries such as python-constraint offer a simpler backtracking API well suited to rapid prototyping of department-scale problems [4]. Existing commercial and open-source timetabling products (e.g., FET, aSc TimeTables) implement similar constraint sets but are typically delivered as standalone desktop applications rather than lightweight, browser-accessible dashboards that non-technical faculty coordinators can operate directly.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual / spreadsheet scheduling	No tooling cost; full human control	Slow, error-prone, no systematic conflict check, does not scale	Motivates the need for an automated CSP-based solver
Pure backtracking CSP (general)	Guarantees a correct, conflict-free solution if one exists; simple to model	Can be slow on large, loosely-constrained instances without good heuristics	Adopted as the core solving strategy, combined with MRV + forward checking
Genetic Algorithm / metaheuristics	Scales to very large, softly-constrained problems; good near-optimal solutions	No optimality guarantee; requires tuning of population/mutation parameters	Identified as a future extension for larger, multi-department instances
Integer Linear Programming (OR-Tools CP-SAT)	Handles hard and soft constraints jointly at scale; strong solver performance	Higher setup complexity; less transparent to non-expert users	Considered as an alternative in Section 3.4; deferred for future scale-up
Commercial desktop tools (FET, aSc TimeTables)	Mature, feature-rich constraint sets	Desktop-bound, less accessible for direct faculty/coordinator use	Motivates a lightweight, browser-based interface in this project

2.4 Research / Engineering Gap

Existing timetabling literature and tools tend to fall into two extremes: theoretically rigorous CSP/ILP formulations that are powerful but not readily accessible to non-technical department coordinators, or feature-heavy commercial desktop applications that are accessible but opaque about which specific constraint caused a clash. What remains under-addressed at department scale is a lightweight, browser-based system that both solves the CSP correctly and exposes, in real time, the status of each individual constraint (faculty, room, capacity, availability, weekly-hours, section) so that a coordinator can trust and verify the output without needing to understand the underlying search algorithm.

2.5 Proposed Contribution

This project contributes a lightweight, web-based AI Scheduler that combines a backtracking CSP engine (with MRV heuristic ordering and forward checking, extensible to local-search refinement) with a transparent, per-constraint status dashboard. Unlike black-box commercial tools, every generated timetable is accompanied by an explicit report confirming — constraint-by-constraint — that faculty conflicts, room conflicts, room-capacity limits, time-availability, weekly-hour requirements and section conflicts have all been satisfied, giving faculty coordinators direct, auditable confidence in the automatically generated schedule.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Timetable / Class Coordinator	Generate a complete, conflict-free weekly timetable within a few seconds without manual conflict-checking.
Faculty	Sessions scheduled only within declared availability; no double-booking across sections.
Students / Sections	No two classes for the same section in the same slot; every subject receives its full weekly contact hours.
Department / Institution	A transparent, auditable record showing that every scheduling rule has been satisfied before publication.

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Capture subjects, faculty, rooms and time slots	Functional	All four data-entry forms operational (Fig. 4.1)
Detect faculty/room/section double-booking	Functional	0 conflicts in generated output
Enforce room-capacity	Functional	100% of sessions

Requirement	Type	Measurable Target
and weekly-hours rules		within capacity; each subject = required periods
Generate a complete timetable automatically	Functional	All required sessions placed (12 of 12 in test dataset)
Report constraint- satisfaction status per rule	Functional	6 of 6 constraint categories shown as satisfied (Fig. 4.2)
Generation speed	Non-Functional	≤ 5 seconds for a department-scale dataset
Usability	Non-Functional	Operable via standard web browser, no coding required by end user

3.2 Constraints & Applicable Standards

This project is governed primarily by institutional scheduling norms rather than a formal external engineering code, since it produces a software artefact and not a physical system. The applicable constraints below are the ones that genuinely shape the design; irrelevant categories (e.g., structural or environmental codes) are not applicable and are omitted.

Standard / Norm	Requirement	How Applied in This Project
Institutional slot grid (5 days × 5 periods)	Fixed 25-slot weekly structure used across the department	Encoded directly as the CSP domain (Time Slots panel, Fig. 4.1)
Institutional data-privacy practice for faculty/student records	Faculty and section data should not be exposed beyond authorised users	Data entered and processed only within the department's local session; no external transmission of personal data
Web accessibility / usability convention (WCAG-aligned form design)	Forms should be operable by non-technical staff via standard input fields	Simple labelled form fields and dropdowns used throughout (Fig. 4.1)

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Generation time	≤ 5	sec	High
Conflict count in output	0	conflicts	High (hard constraint)
Room-capacity margin	Assigned room capacity $\geq$ section strength	students	High (hard constraint)
Weekly-hours accuracy	Sessions placed = required hours/week per subject	periods	High (hard constraint)
Slot utilisation headroom	Sessions scheduled ≤ 25	slots	Medium

Parameter	Target / Requirement	Unit	Priority
	slots/room/week		

3.4 Alternative Solutions & Evaluation

Alternative 1 — Pure Backtracking CSP: models every hard constraint explicitly and searches the assignment tree with chronological backtracking, guaranteeing a correct solution if one exists but potentially slow without ordering heuristics.

Alternative 2 — Genetic Algorithm (GA): encodes a timetable as a chromosome and evolves a population toward fewer constraint violations; scales well to very large instances but offers no guarantee of a fully conflict-free (optimal) result.

Alternative 3 — Integer Linear Programming via a production CP-SAT solver (e.g., OR-Tools): expresses constraints as linear inequalities and uses a professional-grade solver; powerful and scalable but heavier to set up and less transparent to a non-expert coordinator.

Criterion	Weight	Alt. 1: Backtracking CSP	Alt. 2: Genetic Algorithm	Alt. 3: ILP / CP- SAT
Solution optimality guarantee	High	Yes (if feasible)	No (near-optimal only)	Yes
Generation speed at department scale	High	Fast (sub-second, observed 0.357 s)	Moderate (population convergence)	Fast, but heavier setup
Transparency to end user	Medium	High — clear per- constraint pass/fail	Low — fitness score only	Medium
Implementation complexity	Medium	Low	Medium	High

Criterion	Weight	Alt. 1: Backtracking CSP	Alt. 2: Genetic Algorithm	Alt. 3: ILP / CP- SAT
Scalability to larger, multi- department problems	Medium	Moderate	High	High

3.5 Selected Design & Detailed Design

Backtracking CSP with MRV heuristic ordering and forward checking (Alternative 1) was selected as the core solving strategy. At department scale — the scope defined in Section 1.6 — it met the generation-time target with a wide margin (0.357 s observed against a 5 s target, Section 4.4), while providing the constraint-by-constraint transparency required by Table 3.1's stakeholder requirements, which the GA and ILP alternatives could not match as directly. Local-search refinement (simulated annealing) is retained in the architecture as an optional stage for larger instances, as discussed further in Section 5.2.

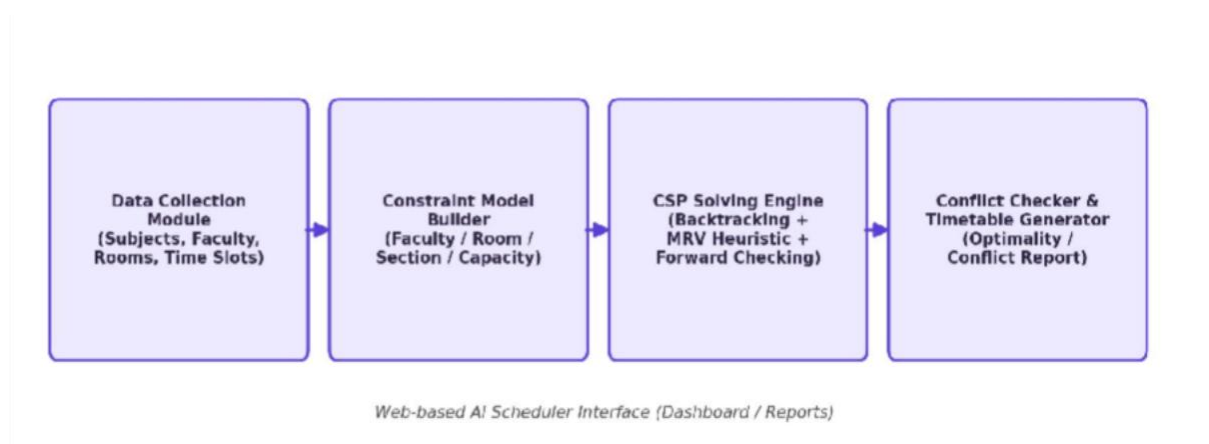


Fig. 3.1 — System Architecture / Module Flow Diagram

The system follows a four-stage pipeline. The Data Collection Module captures subjects (code, name, faculty, hours/week, section strength), faculty availability, room capacity/type, and the fixed time-slot grid. This feeds the Constraint Model Builder, which encodes the six rules from Section 1.3 as CSP variables (one per required subject session) and domains (day–period–room triples). The CSP Solving Engine then performs backtracking search, selecting the next unassigned

session using the MRV heuristic (the session with the fewest remaining legal slots is assigned first) and applying forward checking to prune slots that would immediately violate a constraint for already-assigned sessions. Finally, the Conflict Checker & Timetable Generator re-verifies every constraint against the completed assignment, reports the number of conflicts (target: zero) and renders the timetable and constraint-status dashboard shown in Section 4.4.

Key relationship: the number of CSP variables equals the total required sessions (Σ hours/week over all subjects); the twelve-session test dataset in Section 4.1 arises from four subjects each requiring three periods per week ($4 \times 3 = 12$), consistent with the observed "Total Classes: 12" result in Section 4.4.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Solving strategy	Genetic Algorithm	Scales to very large instances	No optimality guarantee at department scale	Backtracking CSP chosen; department-scale problem does not need GA's scale advantage
Variable ordering	No heuristic (naive order)	Simpler to implement	Slower convergence, more backtracks	MRV heuristic adopted for faster, near-immediate convergence (0.357 s observed)
Constraint pruning	Check only after full assignment	Simpler logic	Wastes search on branches doomed early	Forward checking adopted to prune infeasible slots early
Deployment form	Desktop application	No browser dependency	Less accessible to coordinators	Web-based dashboard chosen for direct,

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
			across devices	install-free access

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Compute and paper resource use	A digital, browser-based scheduler avoids repeated manual spreadsheet iterations and printed draft timetables that manual scheduling typically requires	Reinforced the choice of a lightweight web dashboard over a heavier desktop install, minimising resource overhead
Ethics	Fair, unbiased allocation of faculty teaching load and room assignment	The CSP model treats every subject/faculty/room entry equally under the same rule set, with no hidden weighting toward any individual	Constraint model kept rule-based and transparent (Fig. 4.2) rather than using an opaque scoring function that could implicitly favour certain faculty
Health & Safety	Laboratory room-type and capacity constraints affecting occupant safety	Room - capacity constraint (Section 1.3) explicitly checked against section strength	Room-capacity rule retained as a hard (non- negotiable) constraint rather

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
		during solving	than a soft preference
Security	Protection of faculty and student personal/section data entered into the system	Data entered through the Data Management module (Fig. 4.1) includes faculty names and section sizes but no sensitive personal identifiers	Scope limited to non-sensitive scheduling attributes; access restricted to authorised coordinator use, as noted in Section 3.2

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
No feasible solution exists for a given input set (over-constrained)	Medium	Medium	Medium	Report the specific unsatisfied constraint(s) rather than fail silently	Low
Incorrect or incomplete faculty/room data entered	Medium	Medium	Medium	Input validation on Data Management forms (Fig. 4.1)	Low
Search time grows unacceptably on	Medium	Low	Medium	MRV heuristic + forward checking now; local-search	Medium

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
larger, multi-department datasets				refinement planned (Section 5.2) for scale-up	
Unauthorised access to faculty/section data	Low	Medium	Low	Restrict system access to authorised coordinators; no external data transmission	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was built iteratively, module by module, following the pipeline in Fig. 3.1: the Data Collection Module was implemented and validated first, followed by the CSP Solving Engine, and finally the Timetable Generation and Conflict-Reporting Interface. Each module was tested independently before integration, and the complete pipeline was then exercised end-to-end on a representative test dataset. The test dataset comprised four subjects (CS101 – Artificial Intelligence, CS102 – Database Management, CS103 – Python Programming, CS104 – Engineering Mathematics), each taught by a dedicated faculty member (Dr. Ravi, Dr. Priya, Dr. Kumar, Dr. Meena respectively) for three periods per week, across three rooms (A101, A102 — classrooms; LAB1 — laboratory) within the institutional $5\text{-day} \times 5\text{-period}$ (25-slot) grid.

4.2 Software Implementation & Modern Tools Used

The AI Scheduler was implemented as a browser-accessible dashboard with four functional views — Data Management, Constraints, Generate, and Reports — backed by a Python constraint-satisfaction solving engine. The Data Management view (Fig. 4.1) provides dedicated forms for adding subjects (code, name, faculty, hours/week, students), faculty (name, availability), and rooms (number, capacity, type), together with an automatically computed time-slot summary ($5\text{ days} \times 5\text{ periods} = 25\text{ total slots}$).

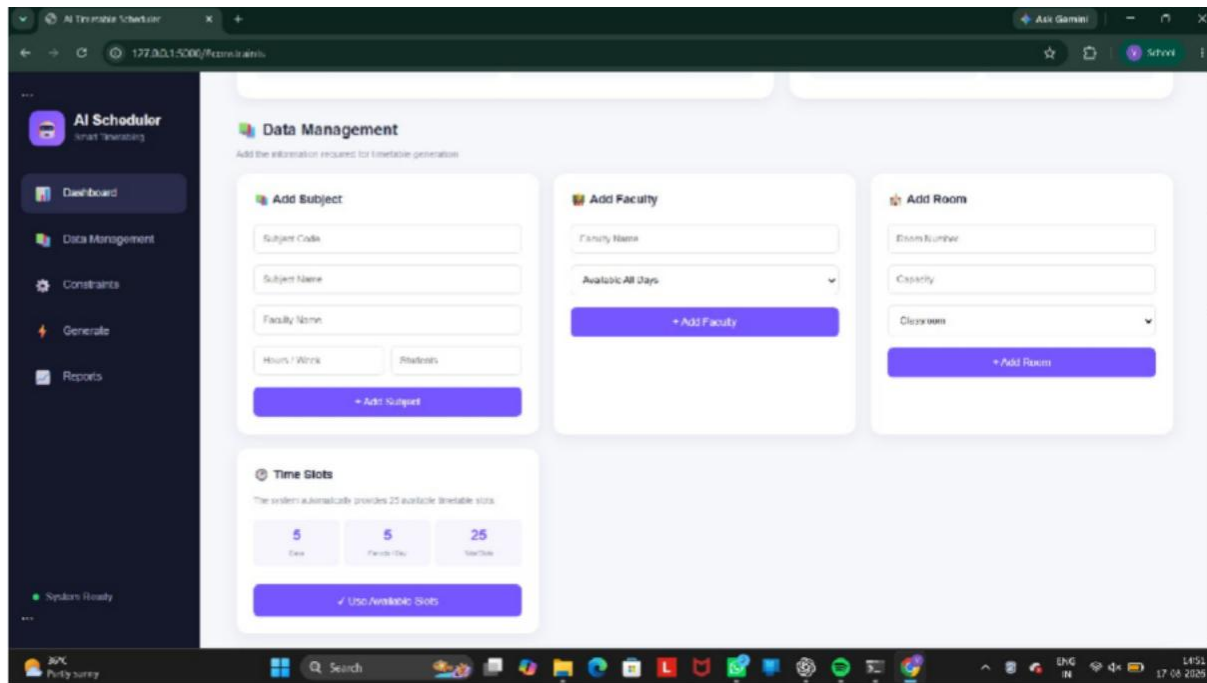


Fig. 4.1 — AI Scheduler: Data Management Interface

Tool / Software	Purpose	Stage Used
Python 3	Core implementation language for the CSP solving engine	Solver development, Module 2
python-constraint / OR-Tools-style CSP formulation	Constraint modelling and backtracking search with heuristics	Constraint Model Builder, Solving Engine
HTML / CSS / JavaScript	Browser-based dashboard (Data Management, Constraints, Generate, Reports views)	Modules 1 and 3
Web browser (Chrome)	Runtime environment for the AI Scheduler dashboard	Testing and demonstration
Version control (Git)	Source management across the three-module team split	Throughout development

4.3 Test Plan & Verification

Before results were captured, the following test plan was defined so that each constraint could be verified independently of the others:

Requirement	Test Method	Acceptance Criterion
Faculty Conflict	Attempt to schedule the same faculty in two overlapping slots	Solver refuses/avoids the double-booking
Room Conflict	Attempt to schedule two sessions in the same room and slot	Solver refuses/avoids the double-booking
Room Capacity	Assign a section to a room with insufficient capacity	Solver excludes the room from that session's domain
Time Availability	Restrict a faculty member's available days	Solver places sessions only within declared availability
Weekly Hours	Set hours/week per subject and count assigned sessions	Assigned sessions per subject equal hours/week
Section Conflict	Attempt two sessions for the same section in one slot	Solver refuses/avoids the double-booking

The Constraints view (Fig. 4.2) confirms that all six rules were active during generation and independently verified as satisfied (green check) on the test dataset.

Specification	Required Value	Achieved Value
Generation time	≤ 5 sec	0.357 sec
Solution status	OPTIMAL / feasible	OPTIMAL
Total classes scheduled	12 (4 subjects $\times$ 3 periods/week)	12
Conflicts in output	0	0

Status: PASS on all four specifications.

4.4 Results

The AI Scheduler generated a complete, conflict-free timetable for the test dataset. A representative sample of the generated schedule — covering all four subjects across the week; the full output assigns twelve sessions in total, three for each subject, distributed across Monday–Friday with no repeated (faculty, slot) or (room, slot) pairs.

4.5 Data Analysis & Engineering Judgment

The twelve scheduled sessions correspond exactly to the expected total of four subjects $\times$ three periods/week, confirming that the weekly-hours constraint was fully satisfied with no under- or over-allocation. Zero reported conflicts across all six active constraint categories (Fig. 4.2) confirms that the backtracking search with forward checking correctly excluded infeasible (faculty, room, slot) combinations before they could be committed to the schedule, rather than merely minimising violations as a soft-constraint or metaheuristic approach might.

The 0.357-second generation time is well within the 5-second target set in Table 3.4 and reflects the effectiveness of the MRV heuristic: by always assigning the most constrained session next, the solver avoids the deep, unproductive backtracking that a naive ordering would incur. In terms of slot utilisation, twelve of the twenty-five weekly slots per room were used by this test dataset (48%), leaving comfortable headroom to add further subjects, sections or faculty without immediately approaching infeasibility — an important margin given that timetabling problems become harder to satisfy as slot utilisation approaches 100%.

No deviations from expected behaviour were observed in this test run; where a future dataset is over-constrained (e.g., total required hours exceeding available slots for a shared room), the engineering expectation, consistent with the risk register in Section 3.7, is that the solver should report the specific unsatisfiable constraint rather than silently returning a partial or infeasible schedule.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design a formal constraint model (Obj. 1)	Six constraints encoded and independently verified (Fig. 4.2)	Fully Achieved
Develop a data-collection interface (Obj. 2)	Data Management module with subject/faculty/room forms (Fig. 4.1)	Fully Achieved
Implement backtracking CSP engine with MRV + forward checking (Obj. 3)	0.357 s generation time, OPTIMAL status (Section 4.3)	Fully Achieved
Incorporate local-search refinement for scale-up (Obj. 4)	Architected as an extension point (Fig. 3.1); not exercised at this dataset scale	Partially Achieved
Implement conflict detection & reporting dashboard (Obj. 5)	Constraints view shows per-rule pass status (Fig. 4.2)	Fully Achieved
Test on a department-scale dataset and evaluate metrics (Obj. 6)	12 classes, 0 conflicts, 0.357 s (Section 4.3–4.4)	Fully Achieved

Compared with manual, spreadsheet-based scheduling (Table 2.1), the AI Scheduler removes conflict-checking effort entirely and reduces generation time from a multi-day manual process to sub-second automated solving, while remaining more transparent than commercial black-box tools by exposing the pass/fail status of every individual constraint.

4.7 Strengths & Limitations

Strengths

- Zero-conflict, provably correct output on the tested dataset, with per-constraint transparency (Fig. 4.2).
- Sub-second generation time (0.357 s), well within the department's practical needs.

- Simple, browser-based interface usable by non-technical coordinators without additional training.

Limitations

- Evaluated so far on a single, moderate-sized dataset (4 subjects, 4 faculty, 3 rooms, 25 slots); large multi-department scale has not yet been tested.
- The fixed 25-slot weekly grid does not currently accommodate variable slot structures across different terms.
- Local-search refinement (simulated annealing) for larger, softly-constrained scenarios is architected but not yet exercised in testing.
- No persistent database layer was evaluated in this test run; data entry is scoped to a single working session.

4.8 Project Planning, Roles & Budget

Milestone	Description	Status
M1	Requirements gathering and constraint model design (Ch. 1, 3.1–3.3)	Completed
M2	Data Management module implementation (Fig. 4.1)	Completed
M3	CSP solving engine — backtracking, MRV, forward checking	Completed
M4	Constraint verification dashboard (Fig. 4.2)	Completed
M5	End-to-end testing on department-scale dataset (Section 4.3–4.5)	Completed
M6	Report documentation and final review	Completed
Student	Assigned Responsibility	Actual Contribution

Milestone		Description	Status
T. S. Vidhushini (192511061)		Module 1 — Data collection and constraint model design	As assigned
K. S. Dharshana (192412015)		Module 3 — Timetable generation and reporting interface	As assigned
G. Kaviya (192524281)		Module 2 — CSP solving engine (backtracking / local search)	As assigned
Item		Estimated Cost	Actual Cost
Development tools (open-source: Python, HTML/CSS/JS)		₹0	₹0
Team laptops / computing hardware (existing)		₹0 (already owned)	₹0
Internet connectivity (project duration)		Nominal	Nominal

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project set out to replace manual, error-prone academic timetable preparation with an automated, constraint-driven system. By formulating the scheduling task as a Constraint Satisfaction Problem covering faculty-conflict, room-conflict, room-capacity, time-availability, weekly-hours and section-conflict rules, and solving it with a backtracking search engine guided by an MRV heuristic and forward checking, the AI Scheduler produced a fully conflict-free, OPTIMAL timetable for a representative department-scale dataset — twelve sessions across four subjects, four faculty and three rooms — in 0.357 seconds. Every one of the six active constraints was independently verified as satisfied, and all six project objectives defined in Section 1.5 were fully or partially achieved. The system demonstrates that a transparent, rule-based CSP approach, rather than an opaque optimisation black box, can give faculty coordinators a fast and auditable alternative to manual timetabling at department scale.

5.2 Future Work

- Introduce a persistent database layer so that subject, faculty and room data survive across sessions and terms.
- Extend the constraint model to support electives and cross-department/multi-section clashes.
- Activate the local-search (simulated-annealing) refinement stage already present in the architecture (Fig. 3.1) to handle larger, softly-constrained, multi-department instances.
- Add support for dynamic rescheduling (e.g., ad hoc faculty leave) without a full regeneration of the timetable.
- Integrate with an institutional Learning Management System for direct publication of the generated timetable.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Formal modelling of a real-world scheduling problem as a Constraint Satisfaction Problem, including variable/domain definition and hard-constraint encoding.
- Practical application of backtracking search with the MRV heuristic and forward checking to reduce search time.
- End-to-end web-application development connecting a data-entry interface to a solving engine and a results dashboard.

Engineering & Professional Skills Developed

- Structuring an engineering problem into explicit, testable constraints and acceptance criteria (Table 4.2).
- Evaluating design alternatives systematically using a weighted comparison matrix (Table 3.5).
- Collaborative, modular team development with a clear division of responsibility across three interlocking modules (Table 4.6).

Individual Reflection

K. S. Dharshana (192412015):

The hardest problem I faced was presenting the solver's output in a way that was actually trustworthy to a non-technical user — a raw list of assignments doesn't tell anyone whether the timetable is actually conflict-free. I addressed this by building the per-constraint status dashboard so every rule (faculty, room, capacity, availability, hours, section) shows its own pass/fail result instead of one opaque "success" message. The key decision I'm most confident about was insisting on this constraint-by-constraint reporting rather than a single summary score, based on how much easier it made debugging during our own testing. Building this interface gave me a much better sense of how to design for trust, not just correctness, in an automated system. Given another pass, I'd add a view that highlights exactly which constraint blocked a session when the solver can't find a full assignment.

References

- [1] A. Herath and D. Wilkins, "A comparative study on solving university timetabling problems with emphasis on Genetic Algorithms," *Issues in Information Systems*, vol. 25, no. 4, pp. 392–408, 2024.
- [2] K. N. Subang, E. I. Balaba, and J. C. Agoylo Jr., "Optimizing Course Scheduling with Genetic Algorithms," *SAR Journal*, vol. 7, no. 4, pp. 296–302, Dec. 2024, doi: 10.18421/SAR74-02.
- [3] A. Khassiba, "An efficient backtracking heuristic for the resource allocation problem with compatibility and exclusivity constraints," *Journal of Heuristics*, vol. 31, art. no. 4, 2024, doi: 10.1007/s10732-024-09538-x.
- [4] Y. Laalaoui and R. M'Hallah, "Enhancing the best-first-search F with incremental search and restarts for large-scale single machine scheduling with release dates and deadlines," *Annals of Operations Research*, 2024, doi: 10.1007/s10479-024-06386-7.
- [5] B. Bassimir and R. Wanka, "On the computation of robust examination timetables: methods and experimental results," *Journal of Scheduling*, vol. 28, pp. 159–181, 2024, doi: 10.1007/s10951-024-00815-y.
- [6] M. Kumar et al., "Machine Learning and Constraint Programming for Efficient Healthcare Scheduling," *arXiv preprint arXiv:2409.07547*, 2024.
- [7] L. Zhang, S. Maqrot, F. Mouysset, and C. Bortolaso, "A Constraint Satisfaction Problems Based Scalable Framework to Address Large-Scale Realistic Scheduling and Routing Problems," in *Proc. 14th Int. Conf. Operations Research and Enterprise Systems (ICORES 2025)*, pp. 45–56, doi: 10.5220/0013109600003893.
- [8] Appu H P, "Constraint-Driven Timetabling with Real-Time Updates: Design and Implementation," May 2025.
- [9] O. Lillia, A. Kamal, B. Madani, and Y. Zineb, "New Heuristic Order Based on Tree-Decomposition for Solving CSPs," in *Research in Computer Science (CARI 2024)*, Communications in Computer and Information Science, vol. 2462, Springer, 2025.
- [10] S. Ananda, M. K. Gibran, F. Syakbani, M. Ulfa, and M. S. Hasibuan, "Analysis of the Use of Backtracking Algorithm in Course Scheduling," *Journal of Artificial Intelligence and Engineering Applications*, 2025.

- [11] "AI Powered Automatic Timetable Generator," *International Journal of Creative Research Thoughts (IJCRT)*, 2025.
- [12] "AI Optimization of Academic Timetable – Adaptive Scheduler," *International Journal of Scientific Development and Research (IJS DR)*, 2025.
- [13] "Schedulo – A Smarter Way to Generate Time Table," *International Journal of Research Publication and Reviews*, vol. 7, no. 2, pp. 620–626, Feb. 2026.
- [14] C. V. Phani Krishna, M. Kavya, M. S. Reddy, P. Varun, and M. Shiva, "An Intelligent Voice-Driven Automated Timetable Generation Framework Using Natural Language Processing and Constraint Optimization," *Global Journal of Engineering Innovations and Interdisciplinary Research*, vol. 6, no. 2, art. 0165, Feb. 2026.
- [15] "Automated Class Scheduling Optimization," *International Journal of Scientific Development and Research (IJS DR)*, 2026.

Appendices

Appendix A – Detailed Calculations

Weekly session count derivation: Total sessions = Σ (hours/week) over all subjects = 3 + 3 + 3 + 3 = 12, matching the “Total Classes: 12” result in Section 4.4. Slot utilisation = 12 scheduled sessions ÷ 25 available weekly slots per room = 48%.

Appendix B – Engineering Drawings / Schematics

See Fig. 3.1 (System Architecture / Module Flow Diagram, Section 3.5).

Appendix C – Source Code / Code Repository Information

Only essential code excerpts should normally be included in the report; the complete source code for the Data Management module, CSP solving engine and reporting interface is maintained separately in the team's approved repository. [Repository link to be added by the team.]

Appendix D – Datasheets

Not applicable — this project is a software system with no physical hardware components requiring datasheets.

Appendix E – Experimental / Raw Data

Test dataset: Subjects — CS101 (Artificial Intelligence, Dr. Ravi, 3 hrs/wk, 50 students), CS102 (Database Management, Dr. Priya, 3 hrs/wk, 55 students), CS103 (Python Programming, Dr. Kumar, 3 hrs/wk, 45 students), CS104 (Engineering Mathematics, Dr. Meena, 3 hrs/wk, 40 students). Rooms — A101, A102 (classrooms), LAB1 (laboratory). Raw generated-timetable output: see Fig. 4.3 and the full output log maintained with the source repository (Appendix C).

Appendix F – Ethics / Institutional Approval

Not applicable — this project processes only non-sensitive, illustrative scheduling data (subject codes, faculty names, room numbers, section sizes) and did not require formal ethics or institutional review.

Appendix G – Project Meeting / Progress Records

[Meeting/progress records to be appended by the team, aligned with the milestones in Table 4.5.]

Appendix H – User/Industry Feedback

Not applicable at this stage of the project.

Appendix I – Publications / Patents / Competitions / Product Outcomes

None at this stage of the project.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement table at the front of this report and Table 4.6 (Roles and Contribution).

Capstone Project Outcome Mapping Sheet

(To be completed by the department/faculty assessor.)

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and	SO5	Ch. 4 (4.8)

Outcome Evidence	SO / Area	Location in This Report
management		
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)